What is Naïve Bayes Algorithm?

Naive Bayes is a classification technique that is based on Bayes' Theorem with an assumption that all the features that predicts the target value are independent of each other.

It calculates the probability of each class and then pick the one with the highest probability.

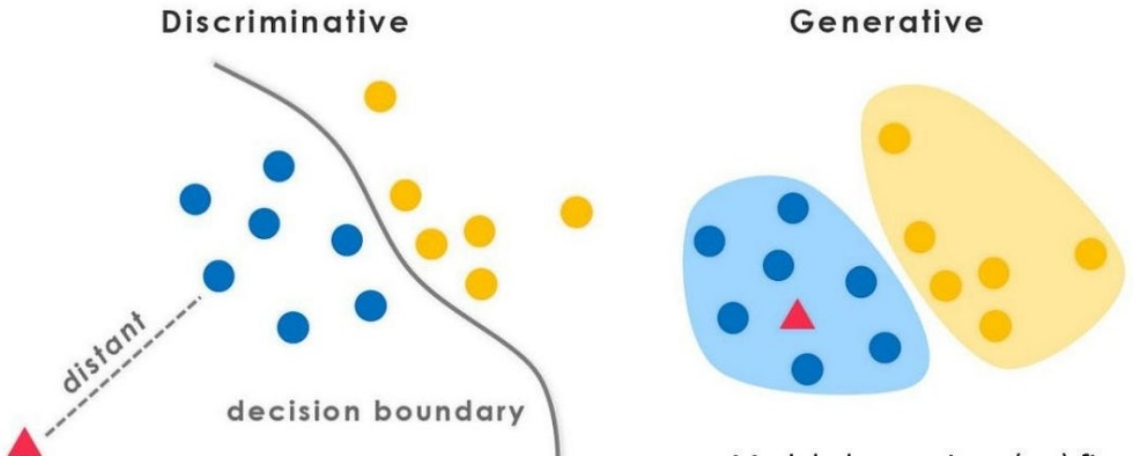
It has been successfully used for many purposes, but it works particularly well with natural language processing (NLP) problems.

What makes Naive Bayes a "Naive" algorithm?

Naive Bayes classifier assumes that the features we use to predict the target are independent and do not affect each other. While in real-life data, features depend on each other in determining the target, but this is ignored by the Naive Bayes classifier.

Though the independence assumption is never correct in real-world data, but often works well in practice. so that it is called "Naive".

Discriminative Model & Generative Model



A discriminative model aims to learn a decision boundary that separates the different classes in the data, while a generative model tries to learn the underlying probability distribution of the data.

Here are some examples to help illustrate the differences between the two types of models:

Logistic regression is an example of a discriminative model. It takes in a set of input features and outputs the probability of a binary classification. For instance, given the features of a flower, logistic regression can predict whether it's a rose or a daisy.

Naive Bayes is an example of a generative model. It models the probability distribution of the features of each class and calculates the posterior probability of each class given the input features. For example, given a set of emails labeled as spam or not spam, a Naive Bayes model can learn the probability distribution of the words in each class and predict whether a new email is spam or not.

Discriminative Model

As is known to us all that machine learning is exactly the technique that predict the target y using features X , which means to compute the conditional probability p(y|X).

Then for the discriminative model, we only consider estimating the conditional probability, which is to establish the discriminant function under the condition of a limited sample, without considering the generative model of the sample, directly studying the prediction model, which can also be understood as in the binary classification problem that

$$p(y=1|X)+p(y=0|X)=1$$

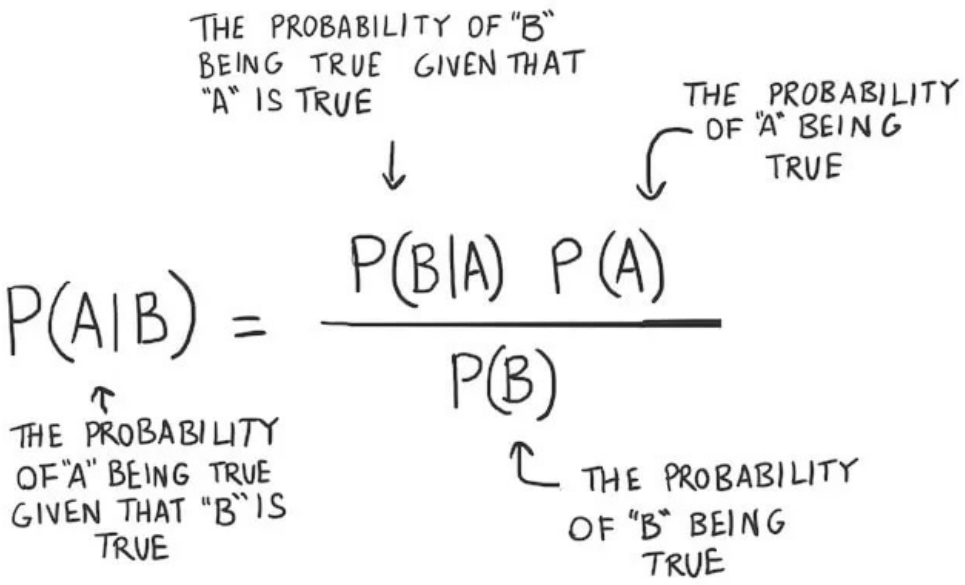
Generative Model

Generative Model

The generative model obtains p(y,X), we need to find the joint distribution probability between X and the target y, and then the big win is to find two joint probability distributions. Take the larger one.

As per wikipedia,In probability theory and statistics, Bayes' theorem (alternatively Bayes' law or Bayes' rule, also written as Bayes's theorem) describes the probability of an event, based on prior knowledge of conditions that might be related to the event.

Lets look at the equation for Bayes Theorem,



Where,

P(A|B) is the probability of hypothesis A given the data B. This is called the posterior probability.

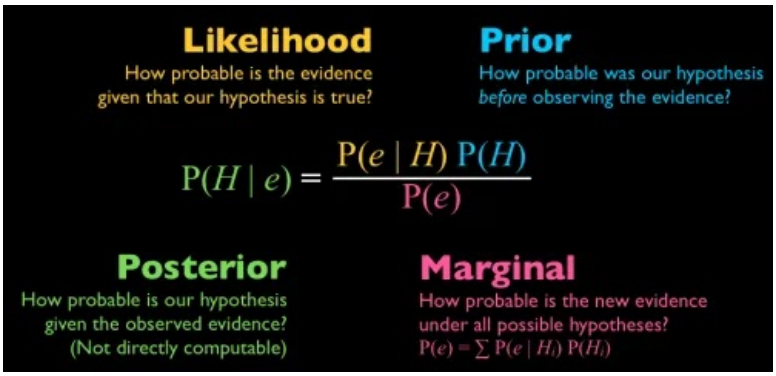
P(B|A) is the probability of data B given that the hypothesis A was true.

P(A) is the probability of hypothesis A being true (regardless of the data). This is called the prior probability of A.

P(B) is the probability of the data (regardless of the hypothesis).

where P(B/A) is conditional probability

$$P(B|A) = \frac{P(A \text{ and } B)}{P(A)}$$



Bayes Theorem !! What exactly is Bayes Theorem?

Bayes Theorem is a very popular mathematical formula that is used to determine the conditional probability of an event, based on prior knowledge of conditions that might be related to the event.

What exactly is Conditional probability?

It is the likelihood of an outcome occurring,given that another event has already occured. For example, two cards are drawn without replacement from a deck of 52 cards. What is the probability that the second card is an ace given that the first card drawn was also ace?

So, P(drawing the first ace) = Total no. of aces / Total no. of cards = 4/52

P(drawing the second ace) = 3/51 (this is because after the first draw we are left with only 3 aces in the deck and the total number of cards also reduces to 51). So, this is what is conditional probability all about. The second event is depend on the occurrence of first one.

How does Naive Bayes Algorithm work?

Let's understand it using an example.Below i have a data set that describes the weather conditions for playing a game of golf. Given the weather conditions, each tuple classifies the conditions as fit("Yes") or unfit("No") for playing golf.

	OUTLOOK	TEMPERATURE	HUMIDITY	WINDY	PLAY GOLF
0	Rainy	Hot	High	False	No
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	Yes
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes
13	Sunny	Mild	High	True	No

Step 1: Convert the data set into a frequency table

Outlook

	Yes	No	P(yes)	P(no)
Sunny	2	3	2/9	3/5
Overcast	4	0	4/9	0/5
Rainy	3	2	3/9	2/5
Total	9	5	100%	100%

Temperature

	Yes	No	P(yes)	P(no)
Hot	2	2	2/9	2/5
Mild	4	2	4/9	2/5
Cool	3	1	3/9	1/5
Total	9	5	100%	100%

Humidity

	Yes	No	P(yes)	P(no)
High	3	4	3/9	4/5
Normal	6	1	6/9	1/5
Total	9	5	100%	100%

Wind

	Yes	No	P(yes)	P(no)
False	6	2	6/9	2/5
True	3	3	3/9	3/5
Total	9	5	100%	100%

Play		P(Yes)/P(No)
Yes	9	9/14
No	5	5/14
Total	14	100%

Step 2: Create Likelihood table by finding the probabilities

Already done in above step

Step 3: Now, use Naive Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of prediction.

Now , assume we need to classify following new instance X, where outlook = sunny, Temperature = cool, humidity = high, and wind = true. So, probability of playing golf is given by:

P(outlook =sunny | play = yes) = 2/9

P(temperature=cold | play = yes) = 3/9

P(Humidity = high| play = yes) = 3/9

P(wind = true | play = yes) = 3/9

P(play = yes) = 9/14

P(outlook =sunny | play = no) = 3/5

P(temperature=cold | play = no) = 1/5

P(Humidity = high| play = no) = 4/5

P(wind = true | play = no) = 3/5

P(play = no) = 5/14

P(X | play = yes).P(play = yes) = 2/9 *3/9* 3/9 *3/9* 9/14 = 0.0053 (calculating likelihood of yes)

(P(X | play = no).P(play = no)) = 3/5 *1/5* 4/5 *3/5* 5/14 = 0.0206 ((calculating likelihood of no)

P(X) = 0.0053 + 0.0206 = 0.0259

P(play = Yes|X) = (P(X | play = yes).P(play = yes)) / P(X) = 0.204

P(play = No|X) = (P(X | play = no).P(play = no)) / P(X) = 0.806

This means that with 0.806 probability ,our prediction that golf will be played is ‘no’.

The method that we discussed above is applicable for discrete data. In case of continuous data, we need to make some assumptions regarding the distribution of values of each feature. The different naive Bayes classifiers differ mainly by the assumptions they make regarding the distribution of P(xi|y).

Let us understand it more better with another example and mathematically intution

Math behind Naive Bays Algorithm

Given a features vector X=(x1,x2,…,xn) and a class variable y, Bayes Theorem states that:

$$P(y|X) = \frac{P(X|y) * P(y)}{P(X)}$$

We’re interested in calculating the posterior probability P(y | X) from the likelihood P(X | y) and prior probabilities P(y),P(X).

Using the chain rule, the likelihood P(X | y) can be decomposed as:

$$P(X|y) = P(x_1, x_2, \ldots, x_n|y)$$

$$= P(x_1|x_2, \ldots, x_n, y) * P(x_2|x_3, \ldots, x_n, y) \ldots P(x_n|y)$$

but because of the Naive's conditional independence assumption, the conditional probabilities are independent of each other.

$$P(X|y) = P(x_1|y) * P(x_2|y) \ldots\ldots P(x_n|y)$$

Thus, by conditional independence, we have:

$$P(y|X) = \frac{P(x_1|y) * P(x_2|y) \ldots\ldots P(x_n|y) * P(y)}{P(x_1) * P(x_2) \ldots P(x_n)}$$

And as denominator remains constant for all values, the posterior probability can then be:

$$P(y|x_1, x_2, \ldots, x_n) \propto P(y) \prod_{i=1}^{n} P(x_i|y)$$

The Naive Bayes classifier combines this model with a decision rule. One common rule is to pick the hypothesis that's most probable; this is known as the maximum a posteriori or MAP decision rule.

$$y = argmax_y P(y) \prod_{i=1}^{n} P(x_i|y)$$

Let's explain it using an example to make things clear:

Assume we have a bunch of emails that we want to classify as spam or not spam.

Our dataset has 15 Not Spam emails and 10 Spam emails. Some analysis had been done, and the frequency of each word had been recorded as shown below:

	Not Spam	Spam
Dear	8	3
Visit	2	6
Invitation	5	2
Link	2	7
Friend	6	1
Hello	5	4
Discount	0	8
Money	1	7
Click	2	9
Dinner	3	0
Total Words	34	47

exploring some probabilities:

P(Dear|Not Spam) = 8/34

P(Visit|Not Spam) = 2/34

P(Dear|Spam) = 3/47

P(Visit|Spam) = 6/47

and so on.

now assume we have the message “Hello friend” and we want to know whether it is a spam or not.

so, using Bayes’ Theorem

$$P(Not\ Spam|Hello\ Friend) = \frac{P(Hello\ Friend|Not\ Spam) * P(Not\ Spam)}{P(Hello\ Friend)}$$

ignoring the denominator

$$P(Not\ Spam|Hello\ Friend) = P(Hello\ Friend|Not\ Spam) * P(Not\ Spam)$$

But, P(Hello friend | Not Spam) = 0, as this case (Hello friend) doesn’t exist in our dataset, i.e. we deal with single words, not the whole sentence, and the same for P(Hello friend | Spam) will be zero as well, which in turn will make both probabilities of being a spam and not spam both are zero, which has no meaning!!

But wait!! we said that the Naive Bayes assumes that `the features we use to predict the target are independent`. so,

$$P(Hello\ Friend|Not\ Spam) = P(Hello|Not\ Spam) * P(Friend|Not\ Spam)$$

$$P(Not\ Spam|Hello\ Friend) = P(Hello|Not\ Spam) * P(Friend|Not\ Spam) * P(Not\ Spam)$$

$$P(Not\ Spam|Hello\ Friend) = \frac{5}{34} * \frac{6}{34} * \frac{15}{25} = 0.0155$$

now let’s calculate the probability of being spam using the same procedure:

$$P(Spam|Hello\ Friend) = \frac{4}{47} * \frac{1}{47} * \frac{10}{25} = 0.00072$$

Not spam > span so, the message “Hello friend” is not a spam.

now, let’s try another example:

assume the message “dear visit dinner money money money”. It’s obvious that it’s a spam, but let’s see what Naive Bayes will say.

$$P(Not\ Spam|dear\ visit\ dinner\ money\ money\ money)$$
$$= P(dear\ visit\ dinner\ money\ money\ money|Not\ Spam) * P(Not\ Spam)$$

$$P(dear\ visit\ dinner\ money\ money\ money|Not\ Spam) = \frac{8}{34} * \frac{2}{34} * \frac{3}{34} * \left(\frac{1}{34}\right)^3$$
$$= 3.107 * 10^{-8}$$

$$P(Not\ Spam|dear\ visit\ dinner\ money\ money\ money) = 3.107 * 10^{-8} * \frac{15}{25}$$
$$= 1.864 * 10^{-8}$$

$$P(Spam|dear\ visit\ dinner\ money\ money\ money)$$
$$= P(dear\ visit\ dinner\ money\ money\ money|Spam) * P(Spam)$$

$$P(dear\ visit\ dinner\ money\ money\ money|Spam) = \frac{3}{47} * \frac{6}{47} * 0 * \left(\frac{7}{47}\right)^3 = 0$$

$$P(Spam|dear\ visit\ dinner\ money\ money\ money) = 0 * \frac{10}{25} = 0$$

oops!! Naive Bays says that this message is not a spam?!!!

This happened because the word “dinner” does not appear in the spam dataset, so that P(dinner | spam) = 0, hence all other probabilities will have no effect.

This is called the Zero-Frequency Problem.

and to solve that we can use Laplace smoothing.

Laplace Smoothing is a technique for smoothing categorical data. A small-sample correction, or pseudo-count, will be incorporated in every probability estimate. Hence, no probability will be zero. this is a way of regularizing Naive Bayes.

Given an observation x = (x1, …, xd) from a multinomial distribution with N trials and parameter vector θ = (θ1, …, θd), a “smoothed” version of the data gives the estimator:

$$\hat{\theta} = \frac{x_i + \alpha}{N + \alpha d} \qquad (i = 1, \ldots, d)$$

where the pseudo-count α > 0 is the smoothing parameter (α = 0 corresponds to no smoothing).

In [22]:
```
# official documentation of additive smoothning : https://en.wikipedia.org/wiki/Additive_smoothing
```

$$P(dear\ visit\ dinner\ money\ money\ money|Not\ Spam) = \frac{8+1}{34+10} * \frac{2+1}{34+10} * \frac{3+1}{34+10} * \left(\frac{1+1}{34+10}\right)^3$$

$$= 1.19 * 10^{-7}$$

$$P(Not\ Spam|dear\ visit\ dinner\ money\ money\ money) = 1.19 * 10^{-7} * \frac{15}{25}$$

$$= 7.144 * 10^{-8} = 0.7144 * 10^{-7}$$

$$P(dear\ visit\ dinner\ money\ money\ money|Spam) = \frac{3+1}{47+10} * \frac{6+1}{47+10} * \frac{0+1}{47+10} * \left(\frac{7+1}{47+10}\right)^3$$

$$= 0.000000418 = 4.18 * 10^{-7}$$

$$P(Spam|dear\ visit\ dinner\ money\ money\ money) = 4.18 * 10^{-7} * \frac{10}{25}$$

$$= 1.672 * 10^{-7}$$

Now it's correctly classified the message as spam.

Types of Naive Bayes Classifiers

Multinomial:

Feature vectors represent the frequencies with which certain events have been generated by a multinomial distribution. For example, the count how often each word occurs in the document. This is the event model typically used for document classification.

Bernoulli:

Like the multinomial model, this model is popular for document classification tasks, where binary term occurrence(i.e. a word occurs in a document or not) features are used rather than term frequencies(i.e. frequency of a word in the document).

Gaussian:

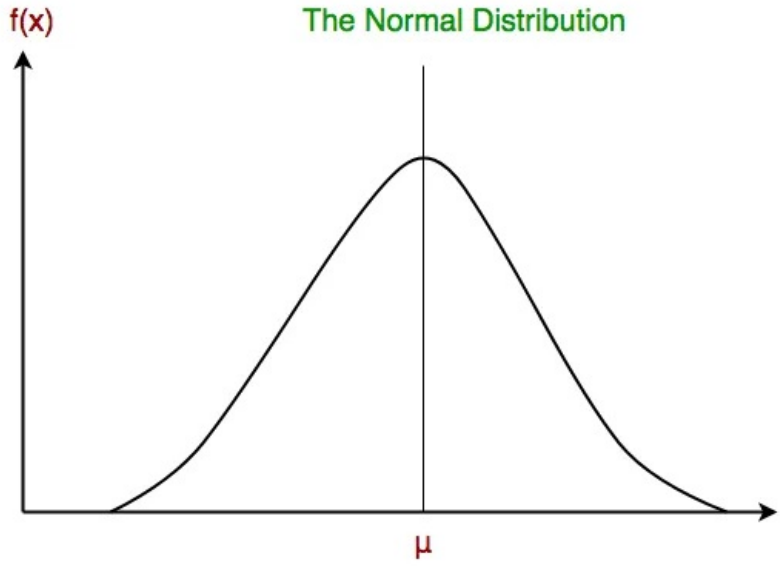
It is used in classification and it assumes that features follow a normal distribution.

Gaussian Naive Bayes Classifier:

In Gaussian Naive Bayes, continuous values associated with each feature are assumed to be distributed according to a Gaussian distribution. A Gaussian distribution is also called Normal distribution. When plotted, it gives a bell shaped curve which is symmetric about the mean of the feature values as shown below:

The likelihood of the features is assumed to be Gaussian, hence, conditional probability is given by:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} exp\left(-\frac{(x_i-\mu_y)^2}{2\sigma_y^2}\right)$$



Pros and Cons for Naive Bayes

Pros:

- Requires a small amount of training data. So the training takes less time.
- Handles continuous and discrete data, and it is not sensitive to irrelevant features.
- Very simple, fast, and easy to implement.
- Can be used for both binary and multi-class classification problems.
- Highly scalable as it scales linearly with the number of predictor features and data points.
- When the Naive Bayes conditional independence assumption holds true, it will converge quicker than discriminative models like logistic regression.

Cons:

- The assumption of independent predictors/features. Naive Bayes implicitly assumes that all the attributes are mutually independent which is almost impossible to find in real-world data.
- If a categorical variable has a value that appears in the test dataset, and not observed in the training dataset, then the model will assign it a zero probability and will not be able to make a prediction. This is what we called the “Zero Frequency problem“, and can be solved using smoothing techniques.

Applications of Naive Bayes Algorithm

- Real-time Prediction.
- Multi-class Prediction.
- Text classification/ Spam Filtering/ Sentiment Analysis.
- Recommendation Systems

Tips to improve the power of Naive Bayes Model

Here are some tips for improving power of Naive Bayes Model:

- If continuous features do not have normal distribution, we should use transformation or different methods to convert it in normal distribution.
- If test data set has zero frequency issue, apply smoothing techniques “Laplace Correction” to predict the class of test data set.
- Remove correlated features, as the highly correlated features are voted twice in the model and it can lead to over inflating importance.
- Naive Bayes classifiers has limited options for parameter tuning like alpha=1 for smoothing, fit_prior=[True|False] to learn class prior

probabilities or not and some other options (look at detail here). I would recommend to focus on your pre-processing of data and the feature selection.

- You might think to apply some classifier combination technique like ensembling, bagging and boosting but these methods would not help. Actually, “ensembling, boosting, bagging” won’t help since their purpose is to reduce variance. Naive Bayes has no variance to minimize.

In [ ]:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js